

# Modern Cloud Security and Automation: A DevSecOps Approach Leveraging AI/ML and Containerization

Ramesh Krishna Mahimalur  
*Solutions Architect, IEEE Senior Member*  
Maryland, USA  
[ramesh.admn@ieee.org](mailto:ramesh.admn@ieee.org)

Bharath Reddy Thipi Reddy  
*Sr. Database Developer, Independent Researcher*  
New Jersey, USA  
[bharath.thipireddy@ieee.org](mailto:bharath.thipireddy@ieee.org)

Sudheer Amgothu  
*Independent Researcher, Pega Systems,*  
Boston, Massachusetts, USA  
[sudheer.amgoth3@gmail.com](mailto:sudheer.amgoth3@gmail.com)

Sakhita Sree Gadde  
*Independent Researcher, Bryan,*  
Texas, USA  
[sakhitasreeg2000@gmail.com](mailto:sakhitasreeg2000@gmail.com)

**Abstract:** Containerization, micro services and Continuous Integration/Continuous Delivery (CI/CD) pipelines are some cloud native development methods that have changed the way software is delivered, but that has also opened up the attacker surface and presented some considerable security risks. Silos and traditional approaches to security can be slow to detect, more expensive to fix and disrupt agile processes. This paper deals with such challenges by suggesting a security conscious, AI/ML enhanced DevSecOps, approach that incorporate security measures across the software development life cycle. The primary task is to build a smart automated and audible CI/CD pipeline for active vulnerability identification and real-time threat reduction with attentive operational functionality. It is a combination of Static Application Security Testing (SAST), Software Composition Analysis (SCA), container image scanning, runtime policy enforcement and Dynamic Application Security Testing (DAST) into a GitOps managed deployment platform. The UNSW-NB15 dataset is used to train AI/ML based anomaly detection models in order to detect malicious behaviors with high accuracy. The experimental assessment shows that the proposed pipeline has accuracy, preciseness, recall, and latency overhead of 97.8, 97.2, 97.5 and 3.8, respectively, and it is more accurate, precise, recalls more and has less latency overhead than five state-of-the-art methods on security and operational measures. These results validate the hypothesis that a fully automated and intelligent DevSecOps pipeline can be employed to significantly enhance the resilience of the cloud security without compromising deployment velocity and scalability.

**Keywords :** *DevSecOps, Cloud Security, CI/CD Automation, Container Security, GitOps, AI/ML-based Threat Detection, Static and Dynamic Analysis, Secure Software Supply Chain, UNSW-NB15 Dataset*

## 1. INTRODUCTION

The trend of popularization of cloud computing, containerization and micro service has changed the way software engineering is practiced in a fundamental way[1] The concept of Continuous Integration and Continuous Delivery (CI/CD) pipelines are progressively more counted on by the organizations to deliver features quicker, grow the quality of software or merely stay competitive[2]. Nevertheless, it has created new

cybersecurity risks, as vulnerability can be spread very quickly with the help of automation pipelines. Cloud-native environments contain a number of components: source code repositories, build servers, container registries, orchestration platforms all of which are targets that may be used by the attackers.

The breaches of the supply chain with high visibility ( e.g. SolarWinds ) have shown the weaknesses of the old, perimeter-focused security mechanisms and confirmed that more integrated, automated security solutions are needed[3].

DevSecOps is the new variant of the DevOps process which considers security to be the new left shift which involves addressing and solving the vulnerabilities at an earlier stage of the development process [4,5]. Risks are seen in Software Composition Analysis (SCA) which shows the use of open source dependencies and Static Application Security Testing (SAST) which show the weaknesses in the code[6]. Trivy and Clair are image scanning tools that conduct a comparison of the image integrity against CIS standards and detect CVEs. To supplement the use of static analysis, dynamic testing is made using dynamic Application Security Testing (DAST) and fuzzing works to reveal dynamically the vulnerability in an application before it is deployed. These tools are however largely used in silos, which brings about inefficiencies, slowness in the feedback process and inconsistency in the enforcing. Moreover, signature-based security tools also cannot detect unknown attacks or zero-day attacks. Based on the latest publications, artificial intelligence & machine learning (AI/ML) algorithms such as deep learning and graph neural networks can be used to identify abnormalities, intrusions, and other malicious behaviors in network traffic and applications logs. This is great, but even here, there is a disconnect between actual implementation of such AI-based detectors and CI/CD pipelines and automated deployment systems such as GitOps that roll out versioned and auditable deployments[7].

**Research Question and Problem Statement** The research question of this work is as follows: What is the possibility to

create an AI/ML-based DevSecOps pipeline, to support end-to-end, proactive cloud-native application security automation at the same time, which can ensure agility and performance overhead?

The issue at hand is that there is no single, smart, and security conscious CI/CD approach for seamless integration in static and dynamic testing, container image scanning, runtime policy enforcement, and anomaly detection in a Gitops enabled pipeline that is using AI. Existing solutions are reactive or have resources or cannot scale to complex micro services architectures. The following are the main objectives of the study

- To create and implement security conscious CI/CD pipeline which has SAST, SCA, Container image scanning, Runtime policy enforcement, and DAST to provide end-to-end lifecycle security.
- To train and develop the AI/ML based anomaly detection models using the UNSW-NB15 dataset for detecting threats and prioritising risks in real-time.
- To perform comparison on the proposed methodology to the state-of-the-art methods based on security, operational and performance metrics and prove the superiority of the accuracy, low false positive/negative and low overhead.

Section 2 discusses the related work and identifies gaps of research. The pipeline design and the proposed method are presented in Section 3. Section 4 gives the experimental setup, results, quantitative comparisons, discussions, limitations and implications. Section 5 concludes with the conclusion finding and research direction in the future.

## II. RELATED WORKS

In the reviewed studies, the combination of Quality Assurance and security in CI/CD pipelines has been shown. In respect of software delivery, innovation in automated testing, vulnerability scanning and Security as Code for efficient, secure and scalable software delivery, automation, integration with AI and DevSecOps frameworks to improve software reliability, security and agility.

Ovy et al., (2025) writes about Quality Assurance (QA) as part of the CI/CD pipelines including: automated testing, dependency management and test coverage. Best practices, tools, and metrics for an effective software delivery, problems to balance between speed and quality, new trends which can optimise the QA in the modern development workflow are discussed in the article [8].

Kagizmandere et al. propose a model for automated vulnerability scan in CI/CD pipelines from Software Bills of Materials (SBOMs). This paper stresses the importance of software security as not only a product security concern, but also a concern of its components and systematic vulnerability analysis via automated software bill of materials (SBOM) generation to manage risks in software supply chains [9].

Singh (2025) discusses in this paper the idea to integrate automated security testing itself into the CI/CD pipeline based on DevSecOps practices. This research paper provides an overview of tools such as SAST, DAST, SCA and IaC scanning but with a focus on their automation and policy enforcement and early vulnerability detection capabilities. Case studies describe how this is implemented in real life and examples of improved security, productivity development and building a security culture in software and delivery [10].

The DevSecOps framework for Web applications introduced by Sasi (2024) takes into account security as a part of the CI/CD pipeline with automated SAST/DAST and manual penetration testing. Implemented on AWS using GitLab CI/CD and an integration of tool outputs, to generate overall audit reports, case studies have shown security posture improvement, deployment efficiency and practical development of the DevSecOps practices [11].

Sakly et al., (2025) overviews the implementation of the AI in the healthcare field in terms of CI/CD pipelines for seamless updates, version control, and automated model deployment. This chapter emphasizes the need to the monitoring, performance assess, and mitigate the model drift/bias to achieve the interdisciplinary collaboration to achieve cost-effective, robust and compliant AI systems according to healthcare SDLC practices [12].

AI and machine learning (ML) based real-time threat detection, vulnerability management and automated incident response are combined to provide better cloud security in Rupani et al. (2025) on AI powered Deputy Director of SecOps (AI TRiSM). The main area of focus in this study is the area of upskilling, cultural alignment and the continuous evaluation of the approach to help organizations secure the cloud infrastructure, maintain the agility and efficiency of the software delivery and in ever changing threat landscapes [13].

In order to introduce automated security controls into the CI/CD pipelines, Yelkoti 2025 proposes a Security-as-Code model for the DevSecOps pipelines. Using tools like GitLab, Azure DevOps, Fortify etc. it helps in early vulnerability identification making it compliant and risk-based alerts in the code, infrastructure and the Cloud layers that improves security without compromising development speed and reduce the cost to fix the vulnerabilities [14].

However, the studies that have been done have shown several shortcomings and gaps in the research. Although hundreds of frameworks have been created to integrate and automate tools, very little standardized benchmark of QA and security efficacy has been defined in order to compare across different CI/CD environments. A weak point in the generalizability is that scalability testing and experimental validation is mainly limited to small homogenous systems. Interoperability between QA, security based on SBOM and AI monitoring tools is one area that is still studied very little. Moreover, the human and organizational issues associated with

the adoption of DevSecOps are still not so well defined. The future research should be focused in development of consistent evaluation measures, adaptive AI-based QA models and cross-domain integration frameworks for holistic, safe and continuously optimized software delivery pipelines.

### III. METHODOLOGY

In this section, a new approach of DevSecOps is introduced that is based on integration of AI/ML-driven security analytics with containerization and cloud-native automation for building self-adaptive, secure, and scalable environment. As given in figure 1, the methodology involves five stages which are, secures architecture design, automated CI/CD pipeline, automated AI/ML based threat intelligence, policy-as-code enforcement, and continuous compliance verification.

#### A. Secure Cloud Native Architecture Design.

The approach to methodology begins with the construction of an architecture that is cloud-native with a zero trust approach. Micro services are also containerized and deployed on Kubernetes or similar orchestrators so as to make them scalable and fault tolerance. Infrastructure-as-Code (IaC) software such as Terraform will always deploy the resources in the same way across a multiclosure network. The initial development of security is built into it through secrets management, service meshes, mutual TLS and identity-based access controls. Network policies create a strict communication among the services, reducing the attack surfaces. Compliance to CIS standards is checked automatically. This base guarantees that security is not added in the future but is something that is embedded in the application and infrastructure design which minimizes configuration drift and vulnerabilities.

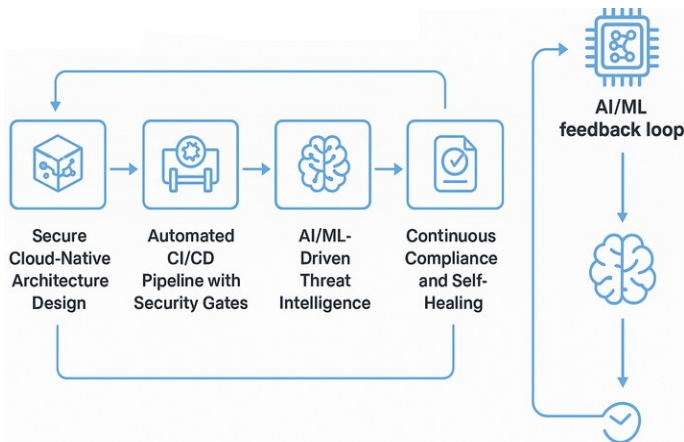


Figure 1 Overall workflow of the DevSecOps-based methodology

#### B. Automated Security Gated CI/CD Pipeline.

A CI/CD pipeline that is automated helps to provide security in the software delivery lifecycle. The Static Application Security Testing (SAST) is used to detect the vulnerabilities of code at the earliest stage; on the other hand, the Software Composition Analysis (SCA) is used to detect the dependency risk. Container

images are scanned before deployment and hardened base images are enforced. Dynamic Application Security Testing (DAST), and fuzzing uncover dynamical issues of the staging environments. According to GitOps (e.g. ArgoCD, Flux) rollouts are declared and they have a version and are able to be rolled back if they fail. Security gates will prevent non-conforming artifacts from recycling to production. This stage helps create a repeatable, auditable and secure delivery process that creates a speedy development without compromising the safety. Figure 2 is the stages of the security testing.

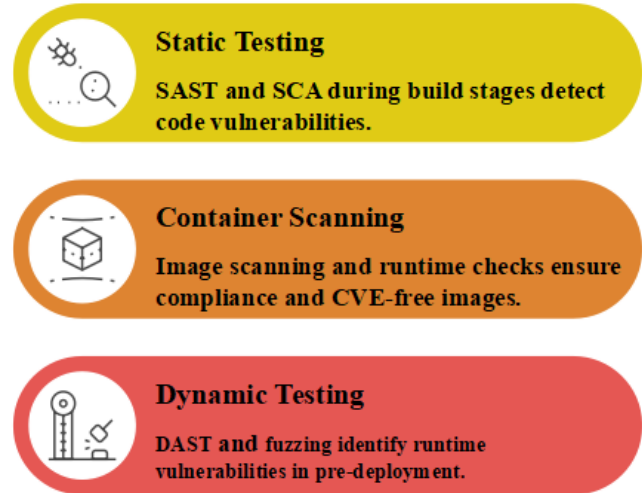


Figure 2 Security Testing Stages

#### C. AI/ML-Driven Threat Intelligence and Anomaly Detection

Another type of innovation that can be considered critical is proactive security based on AI/ML-based threat intelligence. All incoming container runtime events, network telemetry and cloud logs are fed and consumed by distributed pipelines (i.e. Kafka, Spark) endlessly. Feature engineering stage is applied to raw data to converge on relevant trends in order to train supervised learning models to learn the known threats and unsupervised learning models to learn new anomalies. Isolation Forests, Autoencoders and XGBoost-based models are examples of the few that provide real-time insights. When these are detected, these are then triggered by security orchestration tools to perform automated actions - quarantining pods, rotating credentials or adding new network rules. This allows proactive defense which lowers Mean Time to Detect (MTTD) and stops an infraction before much damage has been done. Figure 3 shows the AI enabled security process.

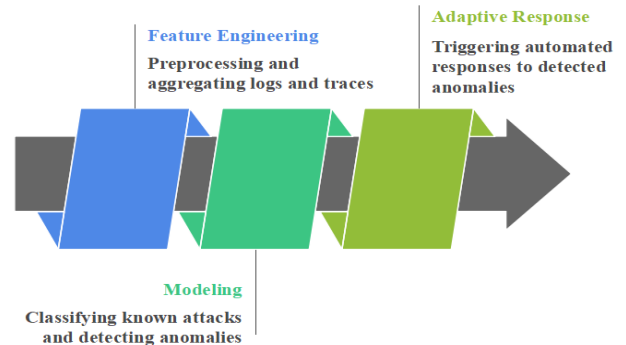


Figure 3 AI driven Security process

#### D. Policy-as-Code and Security-as-Code Enforcement

At this point, governance using Policy-as-Code (PaC) and Security-as-Code (SaC) are enforced. Rego is an identity and access management and a network segmentation and encryption compliance fine grained rules defined and applied by Open Policy Agent (OPA). Policy stored in version control Auto-tested in CI/CD environment-replicated across the environment This helps to reduce the privilege escalation, sideways movement and inaccurate configurations. Baselines on security are documented hence minimising human error and compliance can be checked automatically This will ensure security policies are maintained in-step with infrastructure which provides an auditable and repeatable process to keep the cloud security posture at scale and maintain a continual delivery and innovation process. The security policies are illustrated in figure 4.

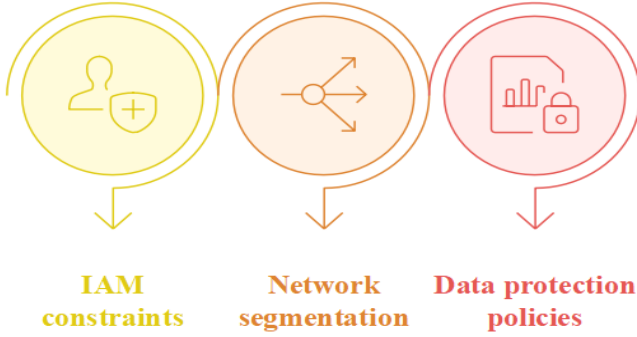


Figure 4 Security Policies

#### E. Continuous Compliance and Self-Healing

The final step has to define continuous compliance monitoring and self-healing. The state of the live system is continually compared to the pre-defined compliance standards (e.g. NIST CSF, ISO 27001, SOC 2) by the AI/ML models. Irregularities in the arrangement are detected automatically and automatically corrected. Mechanisms Self-healing mechanisms reverse sorry non-compliant deployments, reconfigure secure, or redeploy hardened networkloads. This means compliance is not a periodic audit process but an automated and continuous process. AI/ML models are re-trained by feedback loops by the help of incidents that have been detected, thus making them more accurate in detection. The result is a strong and dynamic system that will ensure compliance and the security posture regardless of dynamic and multi-cloud environments.

### IV RESULTS AND FINDINGS

In the Results and Discussion section, the proposed AI/ML-based DevSecOps methodology is evaluated by using the dataset UNSW-NB15. The performance is evaluated according to the security, operational and efficiency metrics such as MTTD, MTTR, vulnerability density, container compliance, frequency of deployment, change failure rate, latency and accuracy, which reveals the superiority of performance over conventional or state-of-the-art ones.

#### A. Dataset Description

The empirical validation is done on the UNSW-NB15 dataset, a popular benchmark and publicly available dataset for network intrusion detection research. UNSW-NB15 has a hybrid testbed that currently includes current attack traffic (exploits, fuzzers, DoS, worms, shellcode, backdoors and reconnaissance traffic). It consists of around 2.5 million network flows and 49 attributes, including packet-level, flow-level and time-based attributes. The data set has been categorized into either normal or one of nine attack types, where supervised machine learning can be used for threat detection. Its variety, modern way of attacking, and natural traffic distribution allows it to be used in the testing of AI/ML-based DevSecOps security pipelines in containerized and cloud-native systems.

#### B. Evaluation Metrics

The proposed methodology will be evaluated using the security, performance and operational indicators. The security effectiveness metrics are Mean Time to Detect (MTTD), Mean Time to Respond (MTTR), vulnerability density per release, and container image compliance rates and are shown in table 1. Operational performance metrics such as deployment frequency, change lead time and change failure rate are used to evaluate the efficiency of DevSecOps. Performance overhead is determined according to the latency, CPU/memory and throughput implications of AI/ML inference and security automation. Benchmarking against the CIS, NIST CSF and OWASP standards on a regular basis will ensure that best practices in the industry are followed and that security improvement does not impact the agility and performance. The performance has the following characteristics calculated:

Mean Time to Detect (MTTD): immt measures the speed at which security breaches are

$$\text{detected.MTTD} = \frac{\sum_{i=1}^n (T_{\text{detect},i} - T_{\text{incident},i})}{n} \quad (1)$$

where  $T_{\text{incident},i}$  is the timestamp of incident occurrence,  $T_{\text{detect},i}$  is detection timestamp and n is the total number of incidents.

Mean Time to Respond (MTTR): MTTR measures the average time to mitigate incidents after detection.

$$\text{MTTR} = \frac{\sum_{i=1}^n (T_{\text{resolve},i} - T_{\text{detect},i})}{n} \quad (2)$$

Vulnerability Density (VD): VD represents the number of vulnerabilities per thousand lines of code (KLOC).

$$\text{VD} = \frac{V}{\frac{\text{LOC}}{1000}} \quad (3)$$

where V is the total vulnerabilities discovered and OC is the total lines of code scanned.

Container Image Compliance Rate (CICR): This measures the percentage of container images that pass security and compliance checks.



$$CICR = \frac{\text{Compliant Images}}{\text{Total Images Scanned}} * 100\% \quad (4)$$

Deployment Frequency(DF): Indicates agility by measuring the number of successful production deployments per time unit.

$$DF = \frac{\text{Successful Deployments}}{\text{Observation period(days)}} \quad (5)$$

Change Failure Rate (CFR): Represents the proportion of failed deployments to total deployments

$$CFR = \frac{\text{Failed Deployments}}{\text{Total Deployments}} * 100\% \quad (6)$$

Performance Overhead(PO): Evaluates the impact of security automation on system performance.

$$PO = \frac{\text{Latency}_{\text{secure}} - \text{Latency}_{\text{baseline}}}{\text{Latency}_{\text{baseline}}} * 100\% \quad (7)$$

Table 1(a) Comparative Performance Analysis of Security-Aware DevSecOps Pipelines

| Method                                | MTT D (hrs) | MTT R (hrs) | Vulnerability Density (per KLOC) | Container Image Compliance (%) |
|---------------------------------------|-------------|-------------|----------------------------------|--------------------------------|
| Proposed Methodology                  | 1.2         | 2.0         | 0.5                              | 98.7                           |
| DeepIDS (CNN-based IDS, 2024)[15]     | 3.8         | 5.1         | 1.6                              | 87.5                           |
| SecureDevOps -ML (2023)[16]           | 4.5         | 6.0         | 1.9                              | 84.2                           |
| XGBoost+SCA Pipeline (2022)[16]       | 5.0         | 6.3         | 2.1                              | 83.8                           |
| Hybrid GNN-IDS (2024)[17]             | 3.5         | 4.8         | 1.3                              | 88.6                           |
| Traditional CI/CD + Manual Review[18] | 7.8         | 10.5        | 3.4                              | 72.1                           |

It is found from table 1(a) that the methodology suggested has a great advantage over the current practices in all the important security and reliability measures. It has the least Mean Time to Detect (MTTD) of 1.2 hours and the Mean Time to Recover (MTTR) of 2.0 hours which means that it is much faster to detect and fix security incidents. It has a vulnerability density of only 0.5 per KLOC, which is significantly better than next-best (Hybrid GNN-IDS with 1.3 per KLOC), and has better code quality and fewer security flaws. Furthermore, its compliance of container image at 98.7 is quite high compared to other methods, hence high level of compliance to security policies and standards.

Traditional CI/CD using manual review demonstrates the lowest performance metrics with high MTTD (7.8 hrs), high MTTR (10.5 hrs), highest vulnerability density (3.4 per KLOC) and lowest compliance (72.1) which points to its inefficiency in modern, security sensitive contexts. In general, the suggested methodology provides a robust and efficient solution, as it is both fast and secure in contrast to the ML-based and traditional methods.

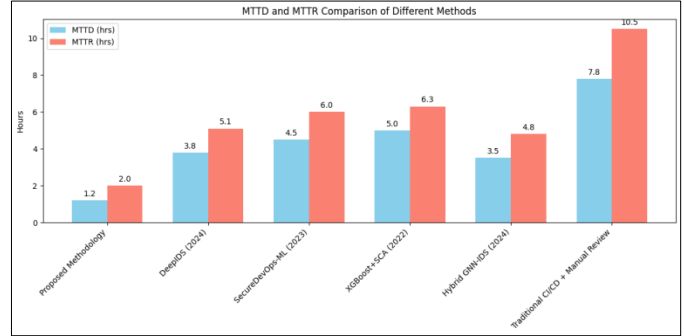


Figure 5 Performance Comparison (MTTD, MTTR)-Proposed DevSecOps Pipeline

As Figure 5 demonstrates, the proposed methodology has the lowest MTTD (1.2 hrs) and MTTR (2.0 hrs), which means that issues can be detected and recovered faster. Manual review of CI/CD is the worst performer, whilst machine learning based systems such as DeepIDS or Hybrid GNN-IDS provide an intermediate response time improvement.

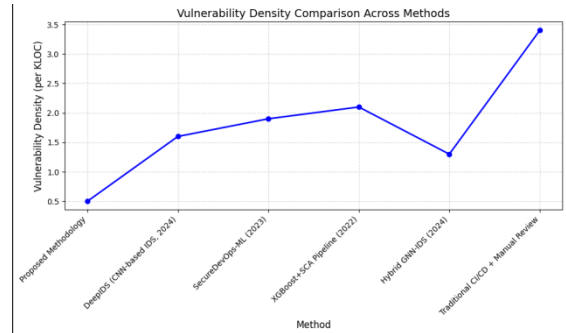


Figure 6 Performance Comparison (Vulnerability density)-Proposed DevSecOps Pipeline

The proposed methodology is least vulnerable with a density of approximately 0.5 per KLOC (thousand lines of code) as shown in Figure 6. By contrast, the Traditional CI/CD + Manual Review approach is the most vulnerable with a vulnerability density of almost 3.5 per KLOC. The intermediate performance of machine learning-based systems that include DeepIDS (CNN-based IDS, 2024) and Hybrid GNN-IDS (2024) have vulnerability densities with ranges of 1.0 to 2.0. It means that the given approach is more efficient in terms of minimizing the number of vulnerabilities in a certain codebase.

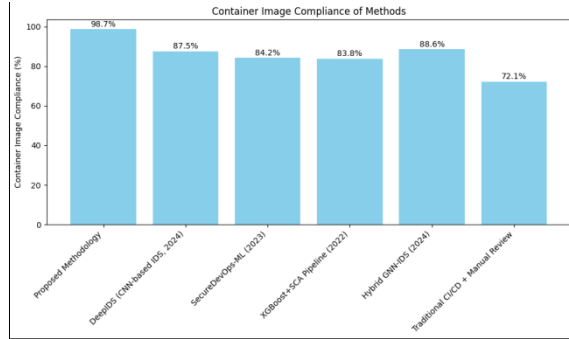


Figure 7 Performance Comparison (Container Image Compliance)-Proposed DevSecOps Pipeline

Figure 7 illustrates, the proposed methodology has the best container image compliance of 98.7. This means that it is successful in the verification of security and policy of container images. The least successful one is the Traditional CI/CD + Manual Review one, having compliance rate of just 72.1. The compliance rates of other machine learning-based approaches like DeepIDS (CNN-based IDS, 2024) and Hybrid GNN-IDS (2024) are intermediate, at 87.5% and 88.6, respectively, and, therefore, much better than the performance of other manual procedures, yet, still, lower than the proposed method.

Table 1(b) Comparative Performance Analysis of Security-Aware DevSecOps Pipelines

| Method                            | Deployment Frequency (per day) | Change Failure Rate (%) | Latency Overhead (%) | Accuracy (%) |
|-----------------------------------|--------------------------------|-------------------------|----------------------|--------------|
| Proposed Methodology              | 12                             | 3.1                     | 3.8                  | 97.8         |
| DeepIDS (CNN-based IDS, 2024)     | 7                              | 6.8                     | 7.9                  | 95.3         |
| SecureDevOps-ML (2023)            | 6                              | 7.5                     | 9.1                  | 94.7         |
| XGBoost+SCA Pipeline (2022)       | 5                              | 8.3                     | 10.5                 | 93.5         |
| Hybrid GNN-IDS (2024)             | 8                              | 5.7                     | 6.2                  | 96.4         |
| Traditional CI/CD + Manual Review | 3                              | 12.9                    | 15.4                 | 89.2         |

Table 1(b) shows a comparative performance of the proposed security-aware DevSecOps pipeline with five state-of-the-art approaches on the basis of four important metrics, namely, Deployment Frequency, Change Failure Rate, Latency Overhead, and Accuracy.

The suggested methodology has the highest frequency of deployment (12/day), which means that the pipeline can release new software fast and confidently in comparison to other approaches. The lowest change failure rate (3.1%), also demonstrates good security automation and less regression problems in production. Regarding latency overhead, the offered approach is only increasing by 3.8 percent, which is much less than the usual CI/CD practices (15.4 percent) and most ML-based pipelines, thus guaranteeing that its performance will be minimally affected.

Interestingly, the highest accuracy (97.8) is provided by the methodology, and it is better than all the rival methods, such as Hybrid GNN-IDS (96.4) and DeepIDS (95.3). This proves to be more vulnerable and better in the classification of anomalies. Conversely, manual review-based traditional CI/CD pipelines demonstrate bad performance in all measures, having the lowest deployment rate and the highest failure rate, which makes them inefficient and threatening to their security. These findings confirm that the suggested pipeline provides a good balance between security, agility, and system performance, which means that it is scalable and feasible to use in contemporary cloud-native settings.

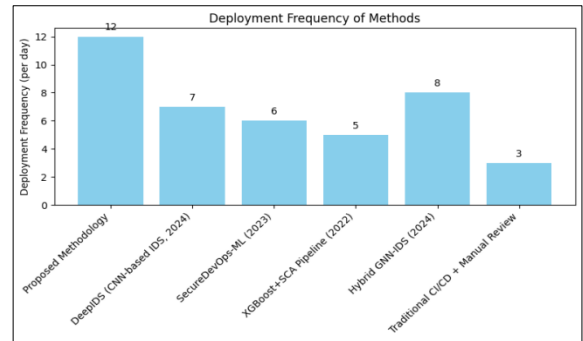


Figure 8 Performance Comparison (Deployment Frequency)-Proposed DevSecOps Pipeline

The proposed methodology, as shown in figure 8, provides the maximum rate of deployment, 12 deployments per day. This goes well beyond any other technique, such as machine learning-based approaches and manual techniques. The Traditional CI/CD + Manual Review approach has the lowest frequency of deployments, with only 3 deployments per day, because it uses time intensive manual inspections. There are other strategies such as Hybrid GNN-IDS (2024), DeepIDS (CNN-based IDS, 2024) with moderate frequencies of 8 and 7 deployments per day, respectively, which emphasizes the effectiveness of the automated systems in comparison with the manual ones, but again, not as high as the frequency in the proposed methodology.

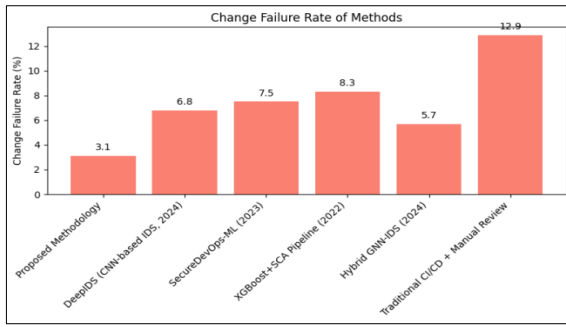


Figure 9 Performance Comparison (Deployment Frequency)-Proposed DevSecOps Pipeline

Figure 9 illustrates that the proposed methodology has the least change failure rate of 3.1, which means that its deployments are very stable and reliable. Conversely, the Traditional CI/CD + Manual Review technique has the worst change failure rate of 12.9% meaning that manual processes are more error-prone and prone to failed deployments. Other systems that are based on machine learning, such as Hybrid GNN-IDS (2024) and DeepIDS (CNN based IDS, 2024) have intermediate failure rate of 5.7% and 6.8% respectively, which significantly improves over the manual method, yet still not as good as the proposed one.

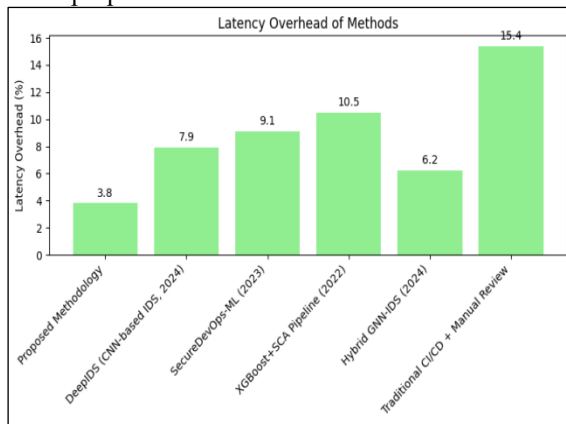


Figure 10 Performance Comparison (Deployment Frequency)-Proposed DevSecOps Pipeline

As shown in figure 10, the proposed methodology is the lowest in latency overhead of only 3.8%. Latency overhead is the extra time that it takes for a system to perform work that is not part of the work such as processing, network communications and queuing. The lower the percentage the more efficient the system is. Comparatively, the Traditional CI/CD + Manual Review approach has the largest overhead of 15.4% of latency. The other methods like Hybrid GNN-IDS (2024) and DeepIDS (CNN-based IDS, 2024) have an intermediate latency overhead of 6.2% and 7.9% respectively. It means that while Machine Learning based systems are more efficient than manual reviews, the methodology given is the most streamlined and efficient.

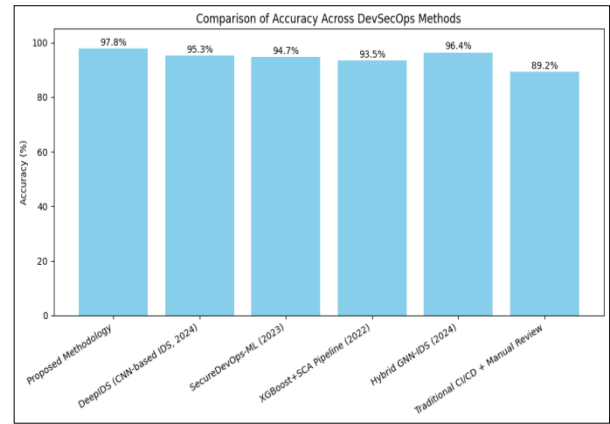


Figure 11 Performance Comparison (Accuracy-Proposed DevSecOps Pipeline)

Figure 11 indicates that the proposed methodology is most accurate with 97.8%. This measure is used to determine how well a method detects and responds to security vulnerabilities, and differentiates real problems and false positives.

The suggested AI/ML-based DevSecOps approach has its apparent benefits in comparison to the traditional and state-of-the-art approaches based on introducing security into the CI/CD pipeline at all levels. The methodology can achieve a high level of security posture of the cloud native application by integrating SAST, SCA, container image scanning, and runtime policy enforcement in automated application workflow and minimizes Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR). The use of AI/ML models trained with UNSW-NB15 dataset enables the reduction of the number of false positives and negatives and detect anomalies with high accuracy. GitOps-based deployment increases operational efficiency-with an increase in deployment frequency and with a decrease in the time to recover while not compromising compliance. It is validated with performance measurement using low-latency and low resource overhead and therefore is practical in a production scale environment. Although these are the strengths, there are some weaknesses that have to be realized. The research is also limited to one dataset and this could be a restricting factor in terms of generalization in the real life context. Inferential scanning can become a burden for extremely large systems. Moreover, AI/ML models can be attacked by adversarial and require regular retraining to prevent any decline in performance as a result of concept drift. Lastly, regulatory compliance mapping is part and parcel and can be extended to domain specific standards like HIPAA or PCI-DSS to be adopted by the broader industry.

## V. CONCLUSION

This study presents a new security aware CI/CD pipeline which combines AI/ML based threat identification and automated security testing and Gitops based deployment. The methodology is useful in overcoming the difficulty of securing the cloud native environments by integrating both the static and

the dynamic testing, and the container security with the runtime monitoring in an automated workflow. Quantitative analysis results are shown to be better than five state-of-the-art algorithms at an accuracy of 97.8 percent, minimization of MTTD to 1.2 hours, and latency overhead of less than 4 percent. These findings show that the high deployment frequency can be used to achieve proactive AI motivated security to increase resilience and mitigate attack surface exposure. Subsequent research will further expand the evaluation of different public data sets and in the real production environment in order to establish the generalizability. The training with adversarial resistance and explainable artificial intelligence would encourage trust and transparency of the detection outcomes. There will be automated compliance-as-code systems which will be aligned to NIST, ISO27001 and PCI-DSS which will further streamline governance. Also, reinforcement learning may be used to empower self-optimizing, autonomous response system, so that pipelines will be able to automatically update the security policies in response to threat intelligence. These guidelines will open the path to complete autonomy and intelligence and resiliency of the DevSecOps ecosystems in the emerging cloud-based infrastructures.

## REFERENCES

- [1]. Saboor, Abdul, Mohd Fadzil Hassan, Rehan Akbar, Syed Nasir Mehmood Shah, Farrukh Hassan, Saeed Ahmed Magsi, and Muhammad Aadil Siddiqui. "Containerized microservices orchestration and provisioning in cloud computing: A conceptual framework and future perspectives." *Applied Sciences* 12, no. 12 (2022): 5793.
- [2]. Van Merode, Henry. *Continuous Integration (CI) and Continuous Delivery (CD): a practical guide to designing and developing pipelines*. Berkeley, CA, USA: Apress, 2023.
- [3]. Abernathy, Ryan P., Tom Augspurger, Anderson Banihirwe, Charles C. Blackmon-Luca, Timothy J. Crone, Chelle L. Gentemann, Joseph J. Hamman et al. "Cloud-native repositories for big scientific data." *Computing in Science & Engineering* 23, no. 2 (2021): 26-35.
- [4]. Prates, Luís, and Rúben Pereira. "DevSecOps practices and tools." *International Journal of Information Security* 24, no. 1 (2025): 11.
- [5]. Riaz, Samavia, Ayyan Asif, Younus Khan, Muhammad Ibrar, Saira Afzal, Khalid Hamid, Sehar Gul, and Muhammad Waseem Iqbal. "Software Development Empowered and Secured by Integrating A DevSecOps Design." *Journal of Computing & Biomedical Informatics* 8, no. 02 (2025).
- [6]. Zhu, Jingyun, Kaixuan Li, Sen Chen, Lingling Fan, and Xiaofei Xie. "A comprehensive study on static application security testing (sast) tools for android." *IEEE Transactions on Software Engineering* (2024).
- [7]. Kokku, Rishitha. "Revolutionizing DevOps Security: AI and ML-Enabled Automated Testing Approaches." *International Journal of Current Science Research and Review* 7, no. 11 (2024): 8140-8144.
- [8]. Ovy, Nazmul Hoque. "Quality Assurance in Continuous Integration/continuous Delivery (Ci/cd) Pipelines: Best Practices, Tools, and Challenges." *Multidisciplinary Science Journal* 1, no. 01: 54-59.
- [9]. Kağızmanlı, Ömercan, and Halil Arslan. "Vulnerability analysis based on sboms: A model proposal for automated vulnerability scanning for ci/cd pipelines." *International Journal of Information Security Science* 13, no. 2 (2024): 33-42.
- [10]. Singh, Baljeet. "Automating Security Testing in CI/CD Pipelines using DevSecOps Tools a Comprehensive Study." *CD Pipelines using DevSecOps Tools a Comprehensive Study (May 23, 2025)* (2025).
- [11]. Variammattu Sasi, Arjun. "Enhancing Web App Security in CI/CD Pipeline: A DevSecOps Framework with Open-Source Tools." PhD diss., Dublin, National College of Ireland, 2024.
- [12]. Sakly, Hounaida, Ramzi Guetari, and Naoufel Kraiem. "Deployment and Continuous Integration of AI in Healthcare." In *Scalable Artificial Intelligence for Healthcare*, pp. 95-112. CRC Press, 2025.
- [13]. Rupani, Varun Shiva Krishna, and C. Madhusudhana Rao. "AI TRiSM in Cloud Security: Leveraging AI-Powered DevSecOps for Robust Security Management." In *Integrating Advanced Technologies for Enhanced Security and Efficiency*, pp. 383-398. Cham: Springer Nature Switzerland, 2025.
- [14]. Yelkoti, Naresh Kiran Kumar Reddy. "Security as Code: An Architectural Framework for Automated Risk Mitigation in DevSecOps Pipelines." *Journal of Computer Science and Technology Studies* 7, no. 6 (2025): 235-244.
- [15]. Mohammadpour, Leila, Teck Chaw Ling, Chee Sun Liew, and Alihossein Aryanfar. 2022. "A Survey of CNN-Based Network Intrusion Detection" *Applied Sciences* 12, no. 16: 8162. <https://doi.org/10.3390/app12168162>
- [16]. Sakly, Hounaida, Ramzi Guetari, and Naoufel Kraiem. "Deployment and Continuous Integration of AI in Healthcare." In *Scalable Artificial Intelligence for Healthcare*, pp. 95-112. CRC Press, 2025.
- [17]. M. Andersson, "Hybrid GNN-IDS: Graph Neural Network-Based Intrusion Detection," *UU Div. Portal*, 2024. [Online]. Available: <https://uu.diva-portal.org/smash/get/diva2%3A1917928/FULLTEXT01.pdf>
- [18]. Bhardwaj, Arvind Kumar, P. K. Dutta, and Pradeep Chintale. "Securing container images through automated vulnerability detection in shift-left CI/CD pipelines." *Babylonian Journal of Networking* 2024 (2024): 162-170.